



S.G. Ganesh

Why C is a Middle-Level Language!

'C' has low-level language features disguised as high-level features. We'll look at two case studies—arrays and functions—to understand what we mean by a 'middle-level language'.

First, look at some features of C functions:

- C doesn't support Pascal-like procedures—the original C always had a return value (void was later added during standardisation; it means the value is returned from a function, but that value is ignored!).
- The standard specifies nothing about the mechanism of argument-passing in C. Normally, the arguments are pushed onto the stack before the function is called. The effect is that the arguments pushed onto the stack are in the reverse order to that given in the function declaration. It is possible to have a variable number of arguments, due to this mechanism—which is close to what is supported at the low level, in contrast to the Pascal argument-passing mechanism.
- C doesn't support nested functions (defining functions within functions). At the physical hardware level, nested functions are usually not supported.
- C supports variable-length argument lists, unlike most other languages.
- When function names are used in C, they decay into pointers. In other words, the name of the function is best thought of as an address, usually in a code segment, where that function's executable code is stored; i.e., the address to which control is transferred when that function is called (directly, or through a function pointer).
- The return values of C functions can be used as l-values (assignable!). For example, given `int *foo()`, we can do `*foo() = 123`.

Now let's cover some features of arrays:

- C arrays reflect the storage of elements in the physical hardware.
- If we just use the name of the array, it decays to a pointer. In other words, the array name refers to the starting address where the actual storage of the array

members begins, and this helps in assigning the array to a pointer of the same element type.

- The array elements are stored in a contiguous storage of bytes.
- No padding (extra spaces) is done between elements of an array.
- The arrays are not self-describing: For example, the length of the array is not remembered in the array itself.
- Given an array `arr` and an index value `i`, all of the following are equivalent: `arr[i]`, `i[arr]`, `arr + i` and `i + arr`! The reason is that array indexing is done by pointer arithmetic on the base address of the array.
- Array indexing starts from 0, and not 1.
- The multi-dimensional arrays are row-major in C (unlike column-major in languages like Fortran). What this means is, multi-dimensional arrays are allocated row-wise instead of column-wise, in a contiguous block of memory. Another implication of this row-major order and contiguous allocation of arrays is that we can treat a multi-dimensional array as if it were a single-dimensional array (by doing proper arithmetic)!

These implementation-level features are close to how arrays and functions are treated at the physical hardware level. If you've done assembly programming, most of these features will be familiar to you! Still, the features of arrays and functions look very high-level, familiar to us, and easy to use. That is the beauty of C! And that is the reason why C is often referred to as a 'middle-level programming language'. 

About S.G. Ganesh

The author works for Siemens (Corporate Technology). He is the author of the bestseller 'Cracking the C, C++ and Java Interview', published by Tata McGraw-Hill. You can reach him at sganesh@gmail.com

A Computer Magazine that does not talk about Music, Games, Photo albums—Only Business

Read Benefit: A Business Minded Computer Magazine



www.benefitmag.com